

EEE 416 (January 2025)

Microprocessors and Embedded Systems Laboratory

Final Project Report

Section: B2 Group: 06

Digital Notification Board for a Seminar Room

Course Instructors:

- 1 Khairul Islam,
Part-Time Lecturer, EEE, BUET
- 2 Akif Hamid,
Part-Time Lecturer, EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

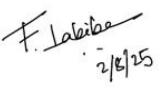
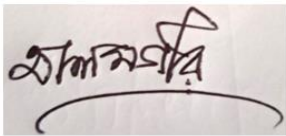
Signature:  2/6/25 Full Name: Fableha Labiba Student ID: 2006119	Signature:  Full Name: 2006120 Student ID: Md. Raihan Siddik
Signature:  02/08/2025 Full Name: Md. Sazidur Rahman Student ID: 2006121	Signature:  Full Name: Md. Alamgir Islam Student ID: 2006122

Table of Contents

Section	Page
1 Abstract	1
2 Introduction	1
3 Design	2
3.1 Problem Formulation (PO(b))	2
3.1.1 Identification of Scope	2
3.1.2 Literature Review	2
3.1.3 Formulation of Problem	2
3.1.4 Analysis	2
3.2 Design Method (PO(a))	3
• Python Script Functionality	3
• ESP32-S3 Master Board Functionality	4
• ESP32-S3 Slave Board Functionality	6
• Communication between Master and Slave with SPI Configuration	7
• Transition from ESP32 Devkit V1 to ESP32-S3	7
• Rationale for Using Two ESP32-S3 Boards	8
3.3 Circuit Diagram	9
3.4 PCB Design	10
4 Implementation	12
4.1 Description	12
5 Design Analysis and Evaluation	13
5.1 Novelty	13
5.2 Design Considerations (PO(c))	13
5.2.1 Considerations to Public Health and Safety	13
5.2.2 Considerations to Environment	13
5.2.3 Considerations to Cultural, Legal and Societal Needs	13
5.2.4 Sustainability in Societal, Environmental, and Economic Contexts	14
5.2.5 Results and Analysis	14
5.3 Limitations of Tools (PO(e))	16
5.4 Impact Assessment (PO(f))	16
5.4.1 Assessment of Societal and Cultural Issues	16
5.4.2 Assessment of Health and Safety Issues	17
5.4.3 Assessment of Legal Issues	17
5.5 Sustainability Evaluation (PO(g))	17
5.6 Ethical Issues (PO(h))	17
6 Reflection on Individual and Team Work (PO(i))	18
6.1 Individual Contribution of Each Member	18
6.2 Mode of Teamwork	19
6.3 Diversity Statement of Team	19
6.4 Log Book of Project Implementation	20
7 Communication to External Stakeholders (PO(j))	22
7.1 Executive Summary	22
7.2 User Manual	22
7.3 Resource Link	23
8 Project Management and Cost Analysis (PO(k))	23
8.1 Bill of Materials	23
8.2 Cost of Prototype Unit	24
8.3 Cost of Mass Production	24
8.4 Timeline of Project Implementation	25
9 Comparing with similar products	26
9.1 Our First Competitor	26
9.2 Our Closest Existing Competitor	27
10 Future Work (PO(l))	27
11 Appendix	28
• Master Code	28
• Slave Code	33

1 Abstract

The project titled "Digital Notification Board for Seminar Room" aims to streamline the display of scheduled and ongoing seminar events using a fully automated system. Users submit meeting details through a Google Form, which are stored in a Google Sheet and processed by a cloud-hosted Python script. This script determines the current status of the seminar room—whether free, scheduled, or occupied—generates a corresponding image using the Pillow library, and uploads a .bin framebuffer file along with a version.txt to a GitHub repository. A pair of ESP32-S3 boards, configured in a master-slave SPI setup, work together where the master downloads and transmits the framebuffer, and the slave renders it to a VGA display using Bitluni's VGA library. This system provides a real-time, visually accessible notification board that enhances transparency and efficient use of seminar room resources while addressing constraints like GitHub's content delivery delay through pre-scheduled rendering logic.

2 Introduction

In academic institutions and corporate environments, efficient utilization of shared spaces such as seminar rooms is essential for maintaining productivity and minimizing scheduling conflicts. Traditional methods like whiteboards, printed schedules, or web-based calendars lack immediacy, physical presence, or automation. This project, titled "Digital Notification Board for Seminar Room", addresses the need for a fully automated, real-time, and physically visible scheduling display system. While the concept appears simple, its implementation presents a complex engineering problem that intersects multiple domains: cloud data synchronization, embedded system communication, SPI interfacing, framebuffer graphics generation, real-time rendering, and wireless networking.

The challenge is further intensified by the limitations of microcontrollers like the ESP32-S3, which have restricted memory and processing power, especially when handling high-resolution image rendering alongside internet communication. Synchronizing time-sensitive updates through GitHub—a platform known to introduce caching delays—adds another layer of complexity. Moreover, ensuring accurate and synchronized rendering using a two-board SPI configuration, while maintaining responsiveness to updated schedules, demands careful timing logic, state detection, and efficient use of hardware resources like PSRAM and DMA.

During the design phase, several alternative solutions were considered. A Raspberry Pi-based setup could offer direct HDMI output and easier access to cloud APIs, but it would be more expensive, less power-efficient, and potentially overkill for this use case. A single ESP32-S3 board was evaluated, but its memory limitations made it impractical for downloading and rendering full-frame VGA content simultaneously. Another option was a web-based scheduling dashboard, which, while accessible remotely, fails to serve as an on-site visual indicator without additional hardware. Ultimately, the chosen architecture—with Google Sheets integration, a cloud-hosted Python script, and dual ESP32-S3 boards for modular handling of download and display—offers a low-cost, scalable, and efficient solution tailored for real-world deployment.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

The scope of this project is to develop an automated digital notification system that displays real-time seminar room schedules using embedded hardware and cloud services. It aims to eliminate manual schedule updates, reduce room booking conflicts, and ensure efficient utilization of seminar spaces through a reliable, low-cost, and scalable solution.

3.1.2 Literature Review

Various digital signage and scheduling systems have been developed using platforms like Raspberry Pi and ESP32, each with trade-offs in cost, complexity, and performance. Raspberry Pi-based systems provide high flexibility but are less suitable for low-cost, power-constrained environments. ESP32 solutions are more affordable but often limited by memory when handling rich graphical content. Few existing works combine real-time cloud-based scheduling with high-resolution VGA output using ESP32-S3, which this project uniquely addresses.

3.1.3 Formulation of Problem

There is a need for a low-cost, automated system to display real-time seminar schedules on a physical screen without manual updates. The problem involves integrating cloud-based data, time-based logic, and embedded hardware to ensure accurate, timely, and clear notifications using resource-constrained microcontrollers.

3.1.4 Analysis

The project requires synchronizing cloud data with embedded hardware while accounting for network delays, memory limitations, and rendering constraints. The use of a dual ESP32-S3 setup—one for data handling and another for VGA display—ensures modularity and efficient resource management. Time synchronization is critical to display updates accurately, especially considering GitHub CDN caching delays. State-based logic in the Python backend minimizes unnecessary updates and optimizes system performance. Overall, the system is designed to be scalable, reliable, and cost-effective for real-world deployment.

3.2 Design Method (PO(a))

Python Script Functionality in the Digital Notification Board Project ^{CO2}

The Python script plays a central role in the Digital Notification Board system by serving as the intelligent backend that interprets meeting schedules and controls what is displayed on the VGA screen in real-time. Hosted on railway.app, the script interacts with Google Sheets, processes time-based logic, and updates a GitHub repository that serves as a content delivery platform for the ESP32-S3 master board.

Key Responsibilities:

Time Synchronization:

- The script fetches the current system time at the beginning of each execution cycle.
- It is designed to wake up at the 0th second of every minute for accurate alignment with meeting schedules.

Data Management:

- Connects to a Google Sheet linked to a Google Form used for scheduling meetings.
- Parses each row to extract meeting date, time, topic, speaker, and other relevant data.
- Automatically deletes rows corresponding to past events.

System State Evaluation:

- Based on the current time and meeting schedule, the system determines one of three states:
 - State A: No upcoming seminar scheduled.
 - State B: A seminar is scheduled but has not started (more than 5 minutes left).
 - State C: A seminar is currently ongoing (less than 5 minutes before or until end).

State Comparison and Behavior Decision:

- Compares the newly determined state with the previous state.
- If state changes, it regenerates the image content.
- If state remains the same, it uses content hashing to determine if anything (e.g., topic, speaker) has changed.
- Avoids unnecessary framebuffer updates to reduce network and processing overhead.

Image Generation:

- Calls specific functions to render the appropriate image using the Pillow library.
- Image is converted to a NumPy array and then to RGB565 format.
- The image is saved as a .bin framebuffer file suitable for ESP32 VGA display.

GitHub Upload:

- Uploads the .bin framebuffer file to a designated GitHub repository.
- Updates a version.txt file containing a version number (modulo 10) and a timestamp (render_time) that tells the ESP when to display the image.

Version Control:

- Implements version tracking by cycling the version number (0–9).
- Ensures ESP32 board fetches the latest framebuffer only when an update is available.

Hashing Mechanism:

- Stores hashes of relevant content (speaker, topic, abstract, etc.) to detect subtle changes even when the state does not change.
- Regenerates and uploads framebuffer only if hash differences are found.

ESP32-S3 Master Board Functionality ^{CO1}

The master ESP32-S3 board handles network communication, file downloading, and SPI data transmission. It periodically checks GitHub for updates and serves as the controller in the master-slave relationship.

Key Responsibilities:

Wi-Fi and Time Sync:

- Connects to Wi-Fi and synchronizes time via NTP (UTC+6).

Version Management:

- Downloads version.txt from GitHub.
- Parses the version number and scheduled render_time.
- Checks for version mismatch to determine if an update is needed.

Framebuffer Download:

- If a new version is detected, downloads the .bin file from GitHub (600 KB).
- Allocates PSRAM memory and stores data.

Render Time Logic:

- Waits until the render_time is reached to counter GitHub CDN caching delay.

SPI Transmission:

- Sends framebuffer in 600 chunks of 1024 bytes each via SPI.
- Uses 5 GPIO pins for handshaking and synchronization (MOSI, SCLK, CS, READY, ACK).

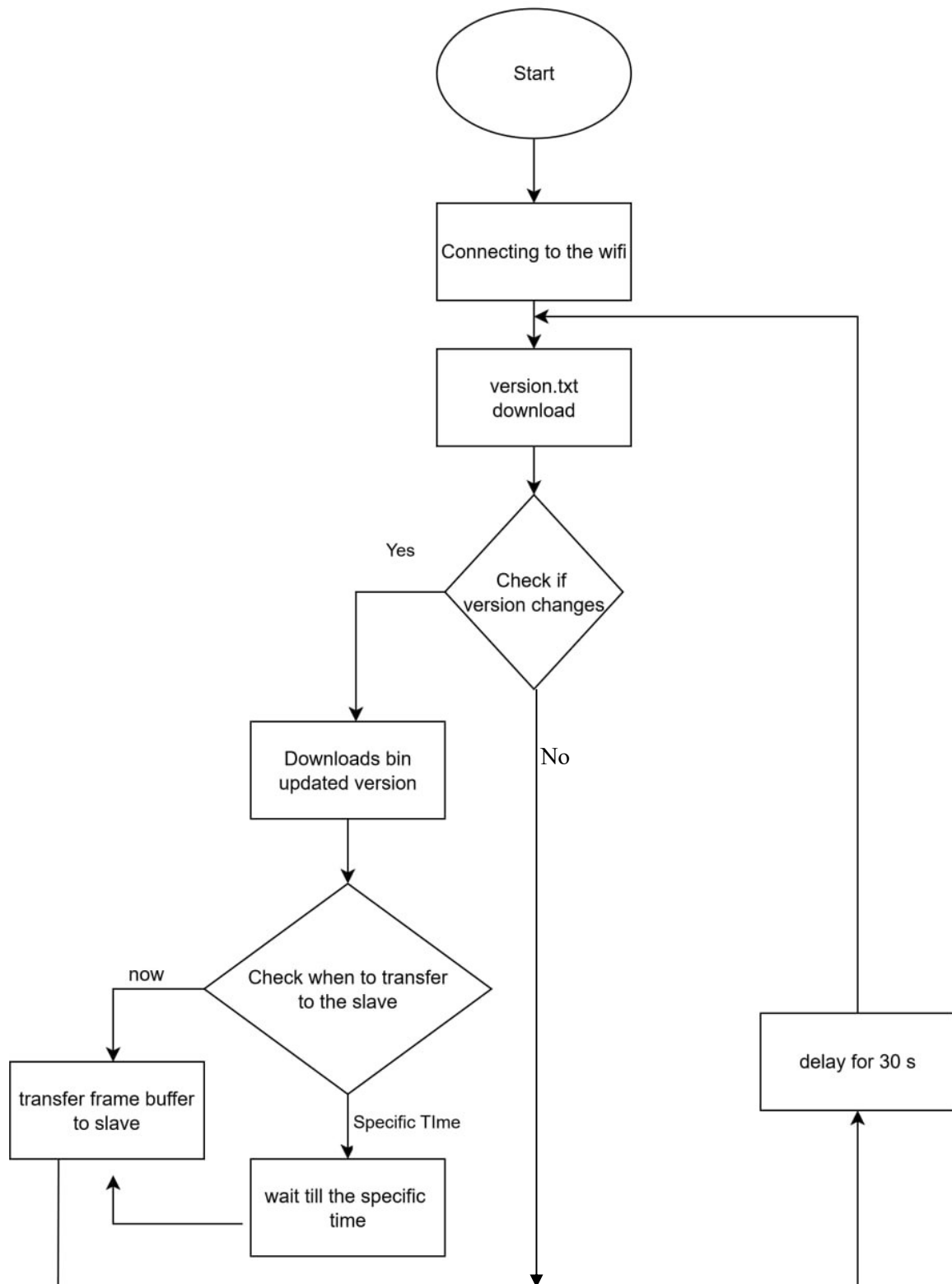


Fig: ESP32-S3 Master Board Functionality

ESP32-S3 Slave Board Functionality ^{CO1}

The slave ESP32-S3 board is dedicated to receiving framebuffer data via SPI and rendering it on a VGA screen using Bitluni's VGA library. It is optimized for graphical output and tightly synchronized with the master.

Key Responsibilities:

SPI Reception:

- Waits for READY signal from master.
- Acknowledges using ACK and receives each chunk using SPI DMA buffer.
- Collects a total of 600 chunks to reconstruct the framebuffer.

VGA Rendering:

- Converts each 16-bit RGB565 pixel into 8-bit RGB values.
- Plots each pixel using `vgaDot(x, y, r, g, b)`.
- Displays the complete image using `vgaShow()`.

Error Handling and Restart Logic:

- Verifies chunk integrity.
- After full image is rendered, restarts if another READY signal is detected.

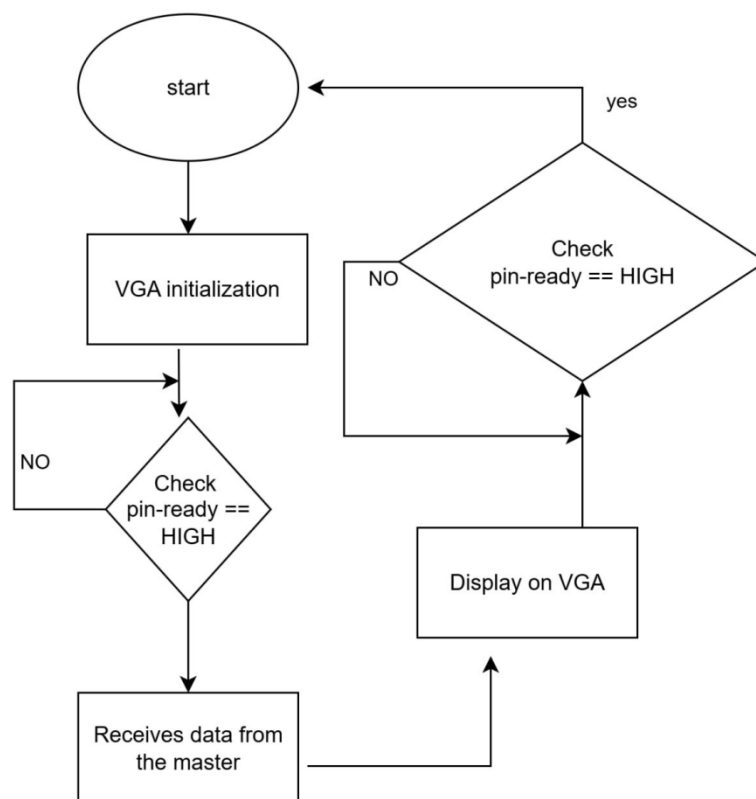


Fig: ESP32-S3 Slave Board Functionality

Communication between Master and Slave with SPI Configuration ^{CO1, CO2}

The SPI communication between the ESP32-S3 master and slave is carefully synchronized using five dedicated GPIO pins: MOSI, SCLK, CS, READY, and ACK. The master and slave use a handshaking mechanism to ensure accurate and reliable transfer of the framebuffer in 600 chunks. Here's how the transfer is carried out:

1. **Master Sets READY High:**

When the master has a complete framebuffer ready to send, it sets the READY pin HIGH to notify the slave that data is available.

2. **Slave Acknowledges with ACK High:**

The slave continuously monitors the READY pin. When it detects a HIGH signal, it sets the ACK pin HIGH to signal that it is ready to receive the data.

3. **Master Starts Transmission (CS Low):**

Upon detecting ACK = HIGH, the master pulls CS LOW to begin SPI data transfer. It then sends the framebuffer chunk over the SPI bus using MOSI (data) and SCLK (clock).

4. **Master Finishes Transmission:**

After the chunk is transmitted, the master sets CS = HIGH and READY = LOW, indicating the end of the current transmission cycle.

5. **Slave Completes Reception:**

Once the slave receives the chunk, it sets ACK = LOW, confirming that it has received the data and is ready for the next chunk.

This process is repeated for each of the 600 chunks until the complete framebuffer is transferred. The synchronization mechanism ensures data integrity, prevents race conditions, and manages timing-sensitive operations essential for accurate VGA rendering.

Transition from ESP32 Devkit V1 to ESP32-S3 ^{CO2}

The initial phase of our project was implemented using the ESP32 Devkit V1 module. However, we encountered significant limitations with this board, particularly in terms of VGA display resolution and memory constraints. The Devkit V1 could only support up to 320×240 resolution with 3-bit color and struggled to operate beyond 200×150 resolution when using 14-bit color. Furthermore, when both Wi-Fi and VGA were initialized simultaneously, the board often crashed due to memory overflow, leaving no room to store the framebuffer. To overcome these issues, we transitioned to the ESP32-S3 N16R8, which offers 8MB of built-in PSRAM. This additional memory allowed us to handle larger framebuffers (e.g., 640×480 with 16-bit RGB565 color) and improved system stability. Despite some conflicts between Wi-Fi and VGA operations in the ESP32-S3, the expanded resources and performance made it a more viable platform for our project's requirements.

Output with ESP32 Devkit V1



3-bit colour, 320x240 resolution



14-bit colour, 200x150 resolution



Jittering for 320x240 resolution, 14-bit colour channel

Rationale for Using Two ESP32-S3 Boards

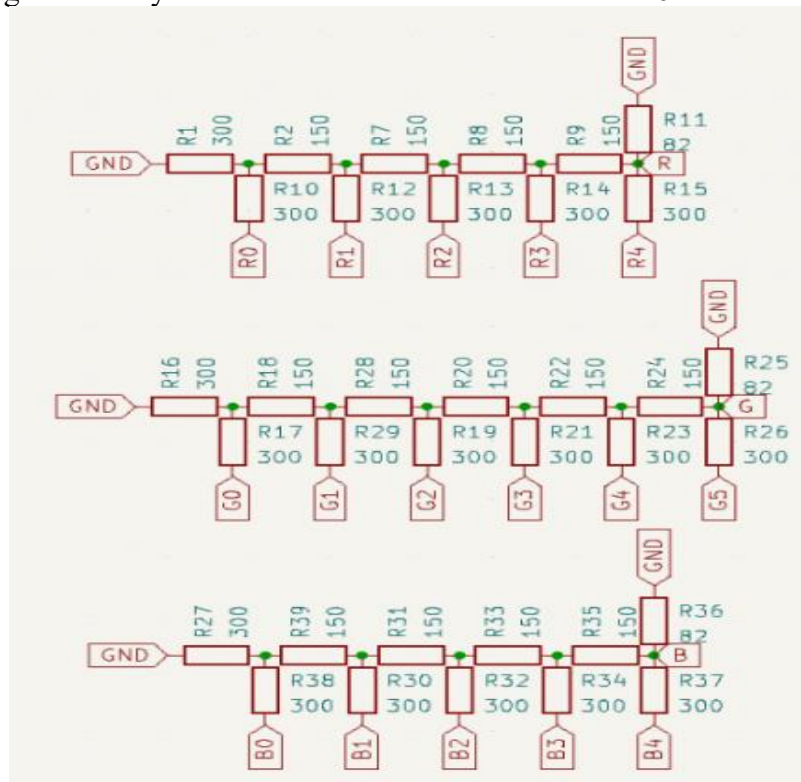
While the ESP32-S3 offers significant improvements in memory and processing power, we encountered critical performance limitations when attempting to run both Wi-Fi communication and VGA rendering on a single board. These two functions often share internal hardware resources, leading to instability, conflicts, and frequent crashes—especially during framebuffer download and display. Moreover, VGA rendering using Bitluni's library demands precise timing and uninterrupted memory access, which Wi-Fi activity can disrupt. To resolve this, we adopted a master-slave architecture using two ESP32-S3 boards: the master handles Wi-Fi and GitHub communication, downloads the framebuffer, and sends it via SPI; the slave is solely responsible for VGA output, ensuring smooth, real-time rendering without interference. This modular approach ensures better performance, reliability, and scalability.



Fig: Distorted images when VGA and WiFi is used at the same ESP32S3

3.3 Circuit Diagram ^{CO2}

Our circuit diagram mainly consists of a DAC ladder circuit for 16 bit color channel.



Here, we have used 220Ω and 470 Ω instead of 150 Ω and 300 Ω resistors.

CO2

We have designed a PCB in easyEDA software.

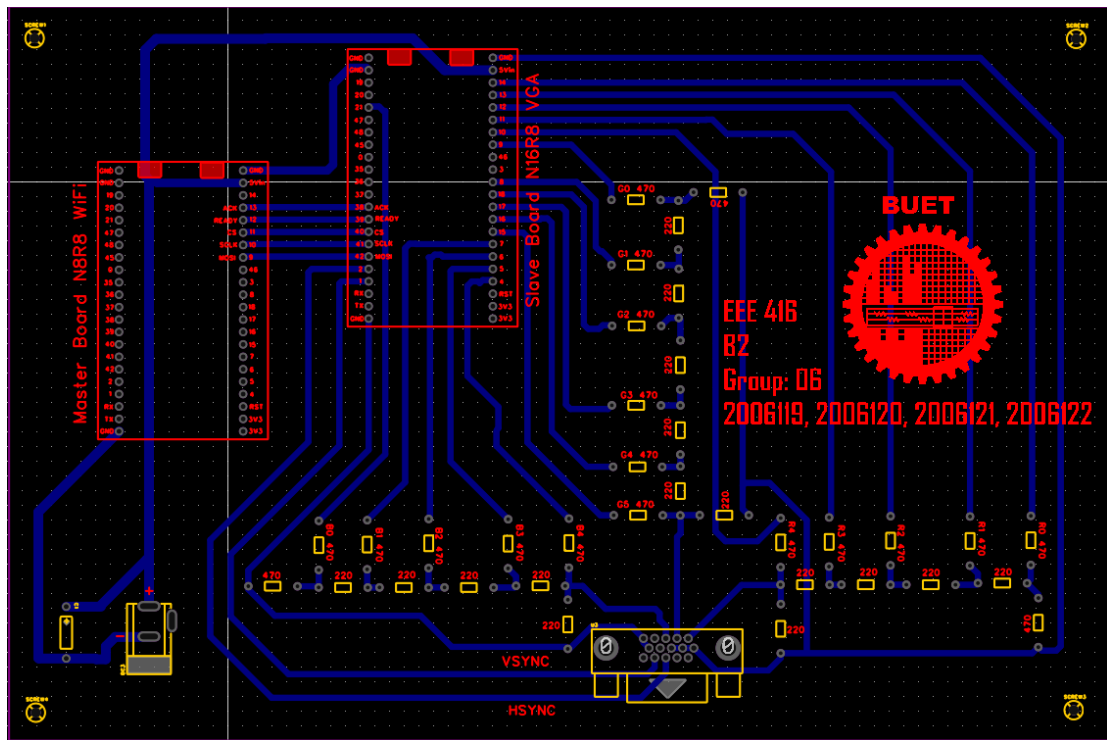


Fig: PCB Designing in easyEDA

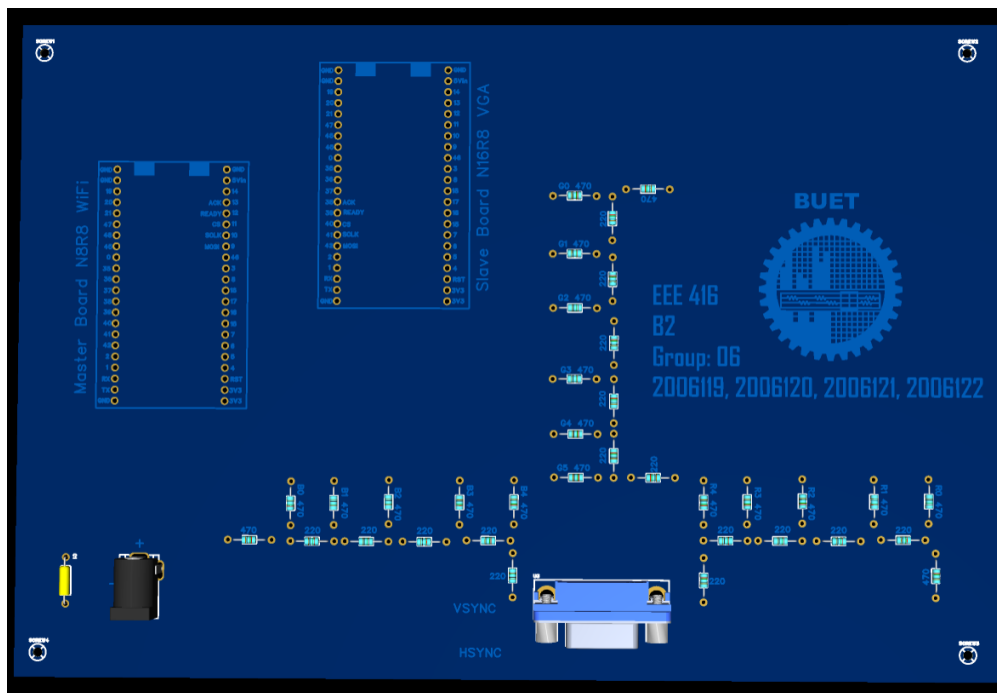


Fig: Our PCB's 3D Layout

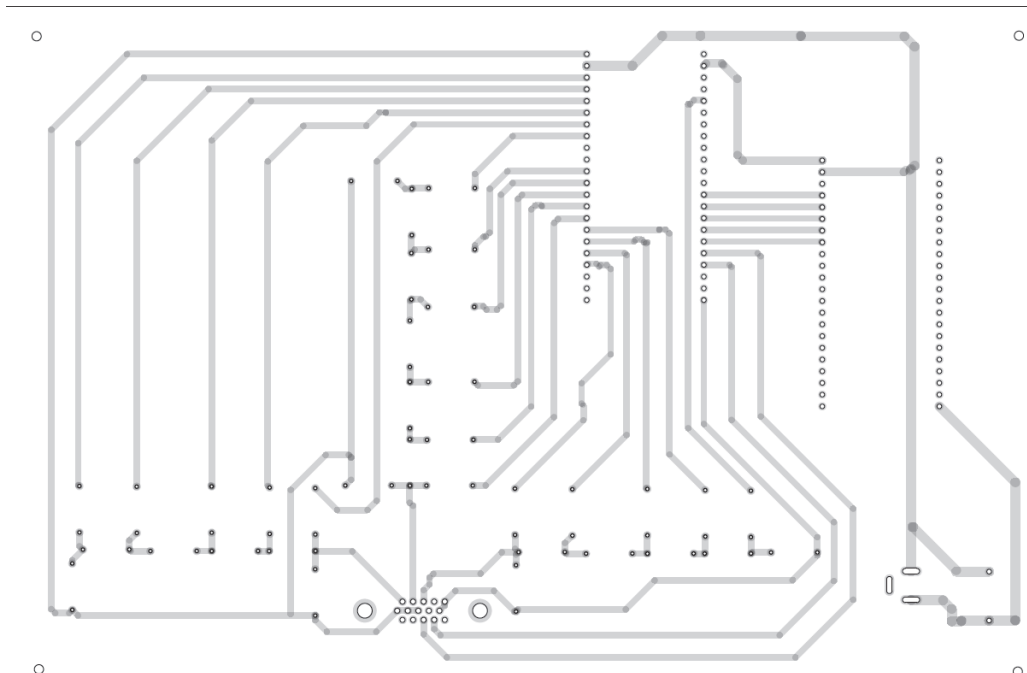


Fig: Our PCB's copper connections

After completing the design, we print the PCB from TechshopBD.

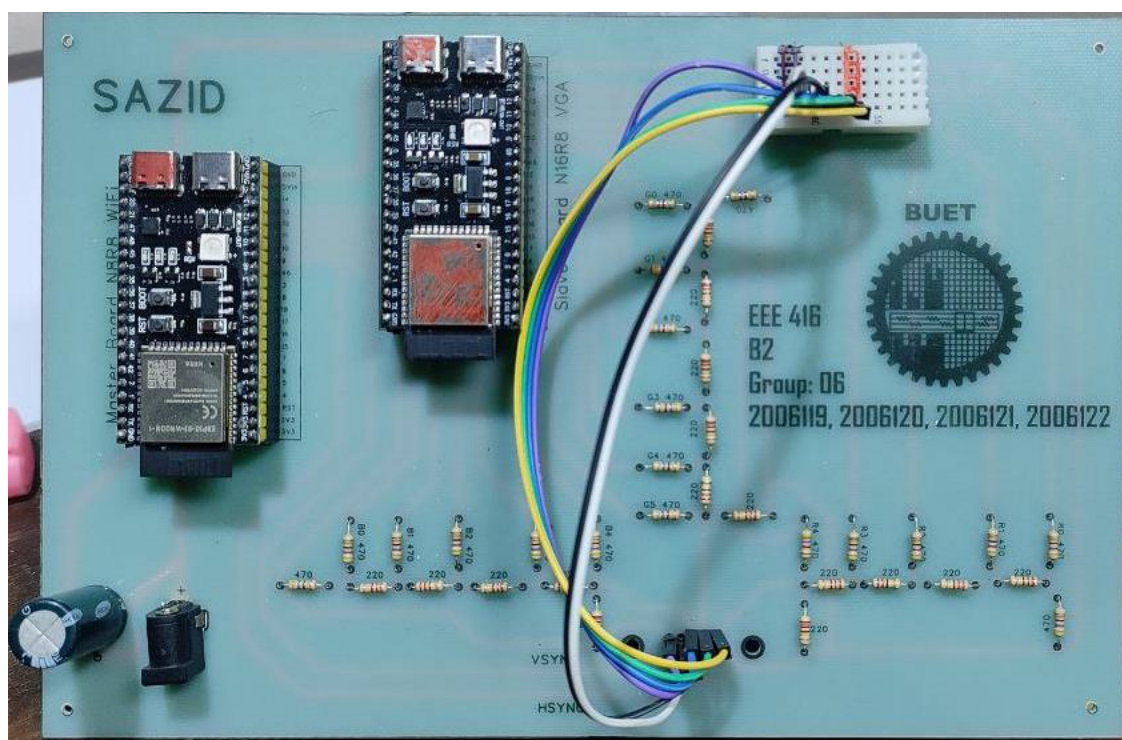


Fig: Our PCB after printing from techshopBD

4 Implementation

4.1 Description ^{CO1, CO2}

The Digital Notification Board system is implemented as a multi-layered architecture combining cloud-based scheduling, automated backend processing, and real-time embedded hardware rendering. Users schedule seminars using a Google Form, which stores responses in a linked Google Sheet. A cloud-hosted Python script (deployed via Railway.app) wakes up every minute to analyze the schedule and determine the system state—whether the room is free, a meeting is upcoming, or a meeting is ongoing.

Based on this logic, the Python script generates a VGA-compatible image using the Pillow library and uploads the corresponding .bin framebuffer file to a GitHub repository. A version.txt file is also updated with a version number and render timestamp to notify the hardware of new content. The master ESP32-S3 board, connected to Wi-Fi, periodically checks for updates on GitHub. If a new version is detected, it downloads the framebuffer and transmits it chunk-by-chunk over SPI to a slave ESP32-S3 board.

The slave ESP32-S3 board is dedicated solely to VGA rendering. It receives the complete framebuffer in 600 chunks and uses Bitluni's VGA library to display the image in 640×480 resolution with 16-bit RGB565 color. The use of two ESP32-S3 boards ensures smooth, conflict-free performance by separating Wi-Fi and display responsibilities. This implementation ensures low-cost, real-time, and autonomous seminar room schedule updates, with minimal manual intervention.

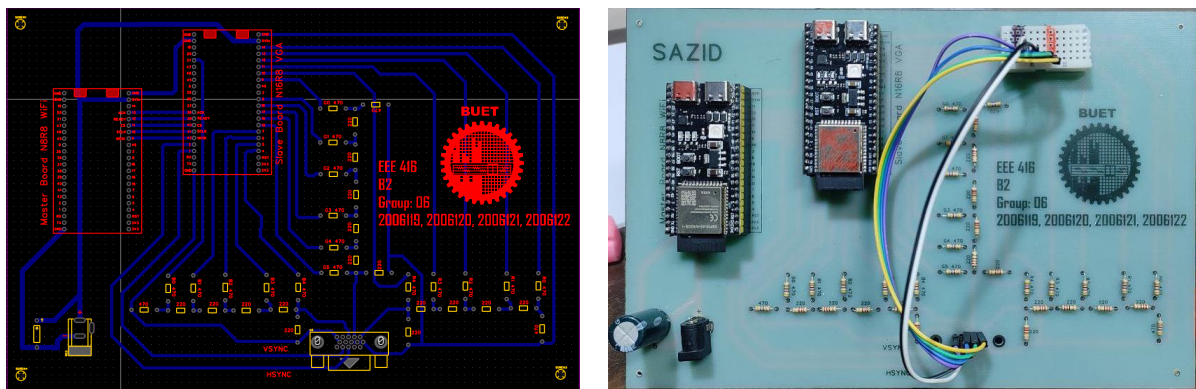


Fig: (Left) PCB Layout and (Right) Implementation of Design

5 Design Analysis and Evaluation

5.1 Novelty

This project introduces a unique, low-cost solution for real-time seminar scheduling by combining cloud automation with embedded systems. Its use of two ESP32-S3 boards—one for downloading and one for VGA rendering—addresses the common hardware conflict between Wi-Fi and display functionalities. Unlike typical systems, it integrates time-aware decision-making, content hashing, and GitHub-based delivery with scheduled render logic to overcome CDN caching delays. The result is a lightweight, autonomous system that updates a physical display without any manual intervention. This modular and synchronized approach is rarely implemented in similar notification systems

5.2 Design Considerations (PO(c)) ^{CO2}

The design prioritizes modularity, reliability, and real-time performance within the constraints of low-cost embedded hardware. The separation of Wi-Fi and VGA functionalities into two ESP32-S3 boards ensures stability and prevents resource conflicts. Timing accuracy, memory usage, and data synchronization were carefully considered to maintain consistent behavior. Additionally, cloud-based backend processing was chosen to offload complex logic from the hardware, simplifying the system and improving scalability.

5.2.1 Considerations to public health and safety ^{CO3}

This project poses minimal risk to public health and safety, as it involves low-voltage embedded electronics and operates in non-hazardous environments such as seminar rooms or academic institutions. The system is designed to be tamper-resistant by limiting user access to scheduling through secure Google Forms. All power supplies and exposed circuits are enclosed or insulated to avoid accidental contact. Additionally, the use of cloud-based control ensures minimal need for on-site maintenance, reducing physical interaction with hardware.

5.2.2 Considerations to environment ^{CO3}

The project is environmentally friendly due to its low power consumption and use of energy-efficient microcontrollers (ESP32-S3). By replacing printed schedules and reducing the need for manual signage, it helps minimize paper waste and contributes to more sustainable communication practices. The system is compact and does not emit harmful substances or generate electronic waste beyond typical embedded devices. Its cloud-based updates also reduce the need for frequent physical interventions, further lowering its environmental footprint.

5.2.3 Considerations to cultural, legal and societal needs ^{CO3}

The system promotes transparency and efficient use of shared public spaces, aligning with societal values of fairness, organization, and productivity. By providing real-time seminar information in an accessible and automated format, it supports inclusive communication and helps prevent scheduling conflicts. The use of simple visual displays ensures that users from diverse cultural and linguistic backgrounds can easily interpret the information. Additionally, the system adheres to legal considerations by using secure platforms like Google Forms and GitHub, ensuring that user data is handled responsibly and access is controlled to prevent unauthorized modifications. Overall, the project enhances user experience in academic and

institutional settings while addressing cultural, legal, and societal needs.

5.2.4 Sustainability in Societal, Environmental, and Economic Contexts ^{CO4}

The system contributes to societal sustainability by improving transparency and reducing manual workload in public resource management. Environmentally, it minimizes paper usage by digitizing schedule displays and uses energy-efficient microcontrollers to keep power consumption low. Economically, it offers a cost-effective solution using affordable hardware, making it scalable and accessible for institutions with limited budgets. Its modular design also supports long-term maintainability and future upgrades without requiring a complete system overhaul.

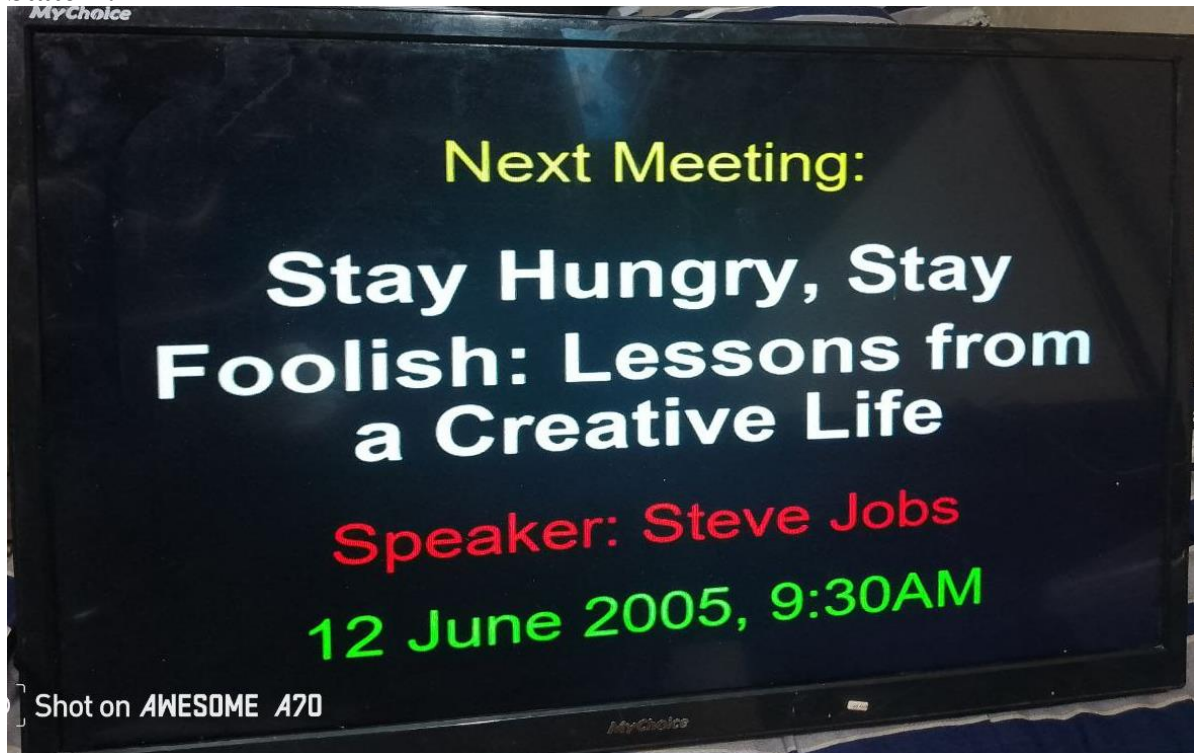
5.2.5 Results and Analysis

Our system produced these results for different states when we are testing it:

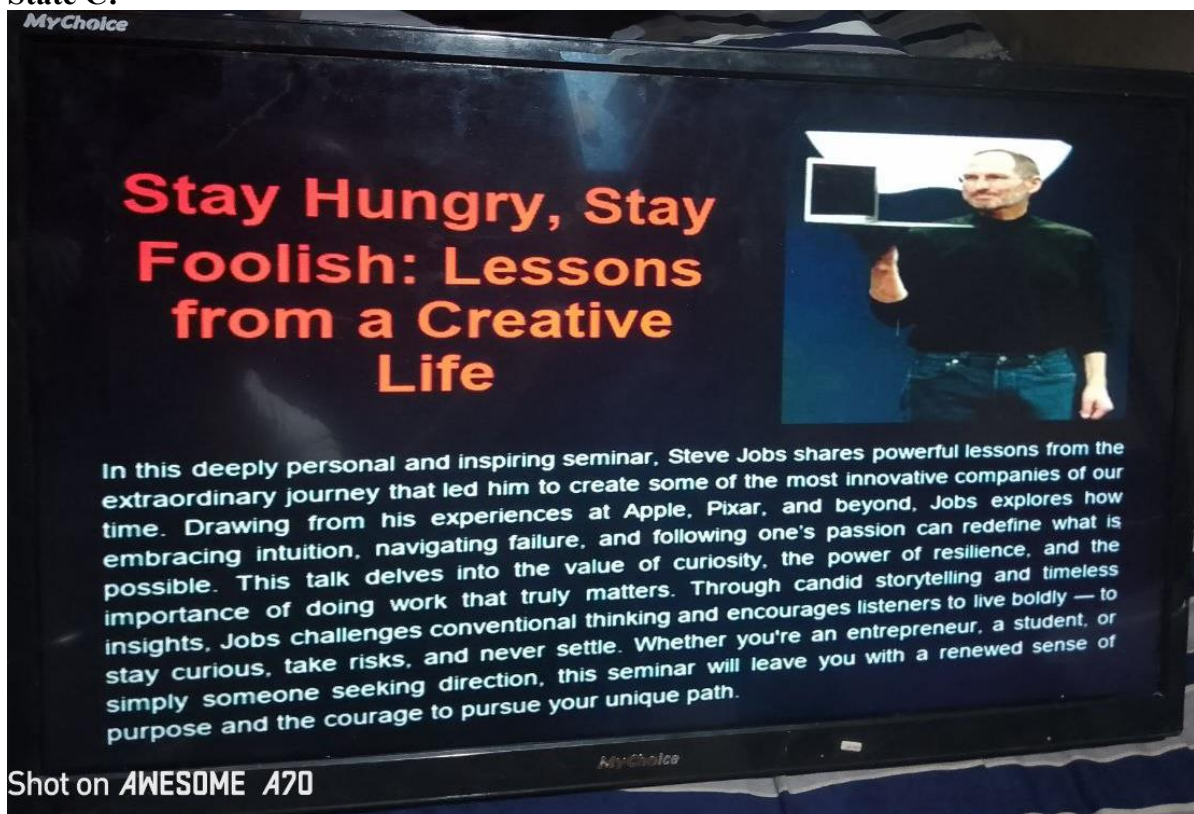
State A:



State B:



State C:



The test results for different system states confirm the correct functionality and smooth coordination between the backend logic and hardware components. In State A, the system successfully detects the absence of upcoming meetings and clearly displays “No Upcoming Meeting” on the VGA screen. In State B, it accurately presents details of the next scheduled seminar—such as the title, speaker, and time—demonstrating proper parsing and rendering of structured content. In State C, the system transitions seamlessly to show the ongoing seminar with the full abstract and speaker image, verifying its capability to handle and render rich content formats. These outputs validate the system’s real-time responsiveness, image generation accuracy, and the reliable communication between the Python script and the dual ESP32-S3 setup.

5.3 Limitations of Tools (PO(e)) ^{CO2}

Several technological limitations were encountered during the development of this system. The ESP32-S3 microcontroller, although more powerful than earlier versions, still has constraints in handling VGA rendering and Wi-Fi communication simultaneously due to shared internal resources and limited real-time multitasking capabilities. This required us to split the tasks across two separate boards. Furthermore, PSRAM availability, while helpful for framebuffer storage, still imposes an upper limit on image resolution and color depth—640×480 with 16-bit RGB565 was the practical upper bound without causing frame drops or jittering.

The use of GitHub as a content delivery server introduces a delay due to CDN caching, which means uploaded files may not be immediately accessible. This affects timing accuracy unless compensated by scheduled rendering logic. On the software side, while the Python script and Google Sheets API are reliable, they may face latency or API rate limits under heavy loads or slow internet connections. Also, floating-point time comparisons and parsing from Sheets may result in minute-level rounding errors if not handled precisely. Despite these limitations, the system performs effectively within its design constraints for real-time seminar display use.

5.4 Impact Assessment (PO(f))

This project contributes to modern academic infrastructure by introducing a low-cost, automated solution for managing seminar room schedules, positively impacting institutional efficiency and communication. By replacing manual signage with a dynamic VGA display, it promotes environmentally responsible practices by reducing paper usage and printing waste. The system is scalable and adaptable to various public settings, including universities, libraries, and corporate meeting rooms, making it relevant in both local and global contexts. Moreover, the integration of cloud-based scheduling, real-time updates, and embedded hardware demonstrates how engineering innovation can address practical societal needs while remaining economically and environmentally sustainable.

5.4.1 Assessment of Societal and Cultural Issues ^{CO3}

The system addresses societal needs by ensuring fair and transparent access to shared resources like seminar rooms, reducing conflicts and miscommunication. Its clear visual output helps users from diverse linguistic and cultural backgrounds easily understand the displayed information, promoting inclusivity. By automating scheduling visibility, it reduces manual workload for staff and supports smoother event coordination. The solution aligns well with the values of efficiency, accessibility, and shared responsibility in academic and professional settings.

5.4.2 Assessment of Health and Safety Issues ^{CO3}

The system is inherently low-risk in terms of health and safety, as it operates using low-voltage electronics within standard academic or office environments. All exposed wiring and circuit components are insulated or enclosed to prevent accidental contact. The automated nature of the system also reduces the need for manual updates or physical handling, minimizing human interaction with electronic components. Overall, the design adheres to safe engineering practices suitable for public installation.

5.4.3 Assessment of Legal Issues ^{CO3}

The system adheres to legal standards by using authorized platforms such as Google Forms, Google Sheets, and GitHub, all of which comply with data privacy and usage policies. User access is restricted to scheduling input only, preventing unauthorized modification or misuse of system content. No sensitive personal data is collected or stored, ensuring compliance with general data protection principles. Additionally, all third-party libraries and APIs used are open-source or properly licensed for educational and non-commercial use.

5.5 Sustainability Evaluation (PO(g)) ^{CO4}

The system demonstrates strong sustainability across environmental, economic, and operational dimensions. By eliminating the need for printed notices, it reduces paper waste and promotes environmentally friendly practices. Economically, it uses affordable microcontrollers and open-source platforms, making the solution cost-effective and feasible for widespread deployment in academic and institutional settings. Its modular design and cloud-based backend allow for easy maintenance, scalability, and future upgrades without major hardware changes. This ensures the system remains functional and adaptable over time with minimal environmental and financial impact.

5.6 Ethical Issues ^{CO3}

Throughout the development of this project, ethical principles were strictly followed to ensure responsible design, data handling, and system transparency. We used only publicly available and open-source tools, libraries, and APIs, and properly credited all third-party resources to avoid intellectual property violations. The system does not collect or display any sensitive personal information, maintaining user privacy and aligning with data protection ethics. Access to scheduling is securely managed through controlled Google Forms, ensuring that only authorized users can submit or modify entries, thereby preventing misuse. During testing, we avoided using real user data and simulated meeting entries to maintain ethical standards. Overall, the project was guided by fairness, accountability, and respect for users and contributors alike.

6 Reflection on Individual and Team work (PO(i)) ^{CO5}

6.1 Individual Contribution of Each Member ^{CO5}

Student IDs	Contribution
20060119	- Preparing code for trial with ESP32 Devkit V1 - Trial with single ESP32S3 - Planning about the system architecture - Implementing SPI configuration - Preparing Python scripts, Google Form, Google Sheet - Hosting Python script to the cloud - Preparing final project presentation slides - Report writing
2006120	- Preparing the first hardware prototypes - Preparing code for trial with ESP32 Devkit V1 - Trial with single ESP32S3 - PCB soldering - Hosting Python script to the cloud - Preparing final project presentation slides - Report writing
2006121	- Components purchasing - Preparing the first hardware prototypes - Preparing code for trial with ESP32 Devkit V1 - Trial with single ESP32S3 - Planning about the system architecture - Implementing SPI configuration - PCB designing - PCB ordering - PCB soldering - Hosting Python script to the cloud - Preparing final project presentation slides - Report writing
2006122	- Preparing code for trial with ESP32 Devkit V1 - Trial with single ESP32S3 - Preparing final project presentation slides - Report writing

6.2 Mode of TeamWork ^{CO5}

Our team adopted a collaborative and role-based mode of teamwork to ensure efficient execution of the project. Each member was assigned specific responsibilities based on their technical strengths and areas of interest, such as hardware development, software scripting, cloud deployment, and documentation. We conducted regular team meetings to review progress, troubleshoot issues, and make design decisions collectively. Communication was maintained through a Telegram group for quick coordination and Google Drive for sharing files and documentation. GitHub was used for version control and maintaining the Python backend code. All members contributed to both the technical and documentation phases, ensuring a well-rounded outcome. Peer feedback and cross-reviewing of each other's work helped maintain quality and coherence. This structured and interactive teamwork approach enabled us to complete the project in a timely and organized manner.

6.3 Diversity Statement of Team ^{CO5}

Although we are classmates from the same academic batch, our team reflects meaningful diversity in several aspects. We come from different districts and educational backgrounds, having studied at different colleges prior to university. Our team includes three male members and one female member, bringing gender diversity and inclusive collaboration. Academically, our majors are not the same—three members specialize in Communication Systems and Processing (CSP), while one is from the Electronics major, offering a broader range of technical perspectives. Our individual areas of interest also vary, covering software, embedded systems, hardware design, and system automation, which allowed us to contribute uniquely to different parts of the project. This diverse mix of backgrounds and skills enriched our teamwork, encouraged creative problem-solving, and fostered a more inclusive and balanced development process.

6.4 Log Book of Project Implementation ^{COS}

Date	Milestone Achieved	Individual Role	Team Role	Comments
14 May 2025	Project proposal presentation and being assigned with VGA rendering task	2006119, 2006120, 2006121, 2006122	All members agreed on using VGA and microcontroller-based plan	Initial task assignment
06 June 2025	First successful trial on bitluni's ESP32 library	2006119, 2006120, 2006121, 2006122	2006120, 2006121 works on both hardware and software, and 2006119, 2006122 works solely on programming the esp32	VGA output achieved
09 June 2025	Rendering custom photo on VGA — stucked due to Wi-Fi and VGA conflict	2006119, 2006120, 2006121, 2006122	All members contributed to the esp32 programming with bitluni VGA library	Led to identifying conflict
10 June 2025	Preparing a draft system architecture	2006119, 2006121	2006121 led architecture planning; 2006119 contributed to design decisions	Prepared a draft architecture with defining the states
12 June 2025	Ordering first ESP32S3	2006121	2006121 handled purchase decision after comparing options	Order placed from RoboticsBD
23–24 June 2025	Trial to implement using single ESP32S3	2006119, 2006120, 2006121, 2006122	All members contributed to the esp32s3 programming	System faced stability issues; led to decision to use two separate ESP32S3 boards.
25–29 June 2025	Purchasing 2 nd ESP32S3 and using separate ESP32S3 with SPI configuration	2006121	2006121 took the purchase decision and ordered the components	Order placed from RoboticsBD
04 July 2025	PCB designing	2006121	2006121 led the design; team reviewed layout and pin mapping	Custom PCB designed for ESP32-S3 and SPI setup
05 July 2025	Ordering PCB print to techshopbd	2006121	Team finalized design and verified before ordering	Order placed successfully
12 July 2025	Receiving printed PCB	2006121	PCB inspected and verified by team	Ready for assembly and testing

Date	Milestone Achieved	Individual Role	Team Role	Comments
16 July 2025	PCB soldering	2006120, 2006121	2006120, 2006121 soldered components and headers; team verified connections	Successful boot and continuity confirmed
16–19 July 2025	Preparing final python script and hosting it to the cloud	2006119, 2006121	2006119 developed the script logic; 2006121 handled deployments	Final backend deployed on railway.app for testing
29 July 2025	Preparing final project presentation slides CO7	2006119, 2006120, 2006121, 2006122	Slides prepared collaboratively with equal input from all members	Final slides included design and implementation.
29 July 2025	Report writing CO7	2006119, 2006120, 2006121, 2006122	All members contributed equally to writing and editing	Report completed with joint effort and review

7 Communication to External Stakeholders (PO(j))

The primary motivation behind this project was to design a system suitable for deployment in the ECE Seminar Room at BUET. Throughout the project, we maintained regular communication with our course instructors who provided valuable guidance on technical and practical aspects of the implementation. We discussed design decisions, feasibility for real-world deployment, and possible integration with the existing infrastructure in the seminar room.

In addition to academic supervision, we coordinated with external vendors like RoboticsBD for components purchasing and TechshopBD for PCB fabrication and used platforms such as railway.app and GitHub for cloud hosting and version control. As a result of this engagement, we are planning to set up the final version of our system in the seminar room during the upcoming term break, marking a real-world application of our work. This report, presentation, and all project materials were also carefully formatted and documented to clearly communicate our design, results, and vision to both technical and non-technical audiences.

7.1 Executive Summary

A group of BUET EEE students has developed a smart digital notification board for the seminar room at the ECE Building. The system automatically displays upcoming and ongoing seminar schedules on a VGA monitor in real-time. Event information is collected through a Google Form and managed via cloud technology. It aims to reduce scheduling confusion and ensure efficient use of the seminar room. Built using ESP32 microcontrollers and open-source tools, the system is cost-effective and fully automated. The team plans to install the system during the next term break for permanent use in the ECE seminar room.

7.2 User Manual

How to Use the Digital Notification Board:

1. Check any upcoming meetings:

- Visit this Google sheet to check the upcoming meetings and free slots.

<https://docs.google.com/spreadsheets/d/1w9C-oExk3cBsDNirHMAB1aJhoZZaNff6hUUJCnnUKw/edit?usp=sharing>

2. Schedule a Meeting:

- Visit the following link to access the seminar scheduling form:
<https://docs.google.com/forms/u/0/d/e/1FAIpQLScYDF5MDlufg-5PfBm6570GBCYUgweulmzE2DCI2YAmzOtWlg/formResponse>
- Fill in the required details, including:
 - Seminar topic
 - Speaker name
 - Date and time

- Abstract and speaker photo URL (optional)
 - Submit the form. A confirmation email will be sent to allow future edits.
4. **Automatic Processing:**
- A cloud-hosted Python script checks the schedule every minute.
 - Based on current time and event status, the system updates the VGA display to show:
 - **No Meeting Scheduled**
 - **Upcoming Seminar**
 - **Ongoing Seminar with Abstract and Speaker Photo**
5. **No Manual Interaction Needed:**
- The display updates automatically without any manual refresh.
 - Only authorized users with the form link can schedule or edit meetings.

This hands-free system ensures timely and accurate seminar updates on the display.

7.3 Resource Link

All the python scripts, ESP32S3 codes, presentation slides, demonstration videos and other resources are uploaded here:

<https://drive.google.com/drive/folders/1kJJJp28hdYYq1eizN-JLAX9iLZ4YLjI?usp=sharing>

All the python scripts, ESP32S3 codes are uploaded here in this **Github** link:

https://github.com/Sazid-121/digital_notification_board_project_eee_416

8 Project Management and Cost Analysis (PO(k)) ^{CO7}

8.1 Bill of Materials

Component	Unit Price
ESP32S3 N16R8 (WROOM1)	650
PCB Board	1300
5V AC/DC power supplier adapter	80
Capacitor 470 μ F	165
Resistors (470 Ω)	2
Resistors (220 Ω)	2
Breadboard Mini Size	45
Connecting Wire	2.50
DC Power Jack Socket Connector	10
VGA Cable	200
Miscellaneous cost	15

8.2 Calculation of Per Unit Cost of Prototype ^{CO7}

Here only the cost for our VGA monitor driver is mentioned

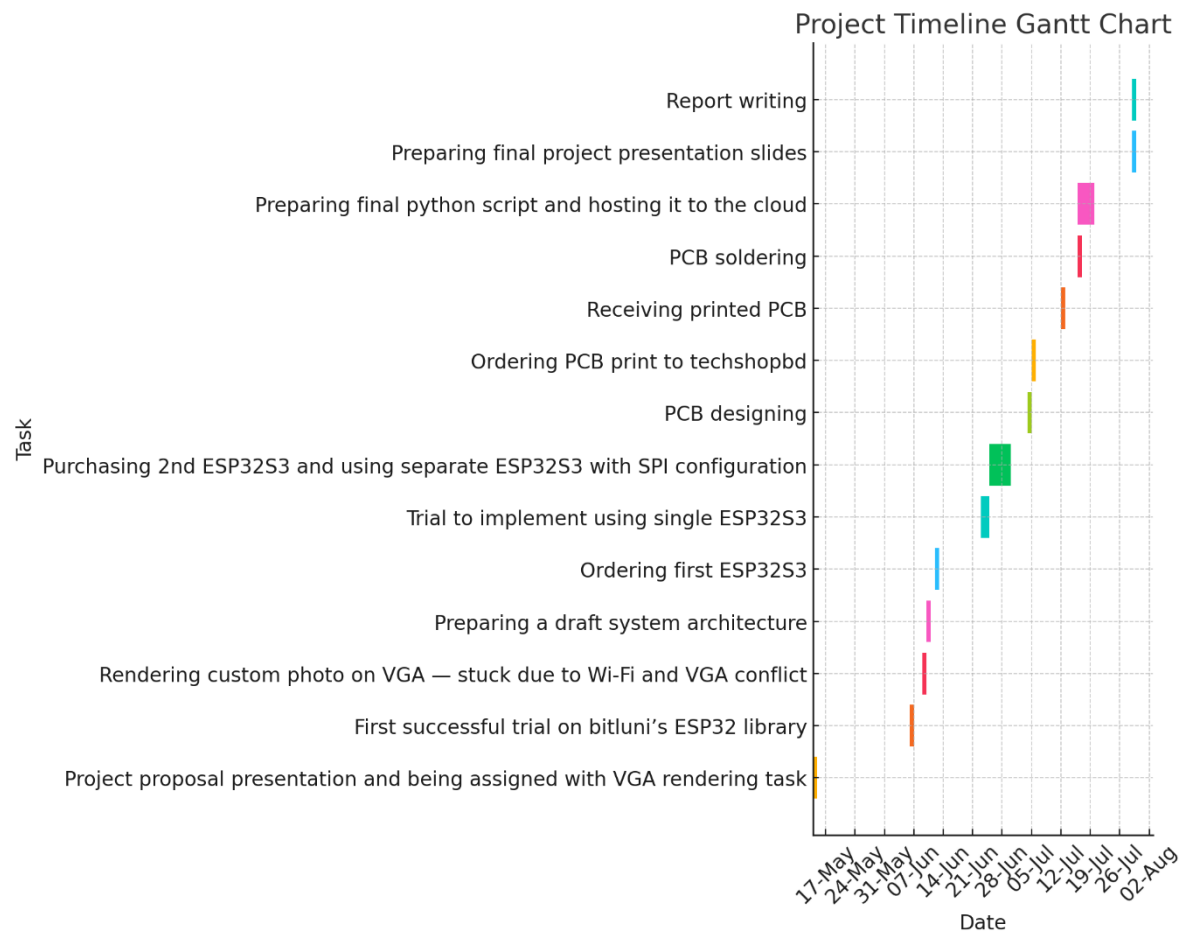
Component	Unit Price	Unit	Total Cost
ESP32S3 N16R8 (WROOM1)	650	2	1300
PCB Board	1300	1	1300
5V AC/DC power supplier adapter	80	1	80
Capacitor 470 μ F	165	1	165
Resistors (470 Ω)	2	19	38
Resistors (220 Ω)	2	16	32
Breadboard Mini Size	45	1	45
Connecting Wire	2.50	6	15
DC Power Jack Socket Connector	10	1	10
VGA Cable	200	1	200
Miscellaneous cost	15	---	15
Total			3200

8.3 Calculation of Per Unit Cost of Mass-Produced Unit ^{CO7}

Here only the cost for our VGA monitor driver is mentioned

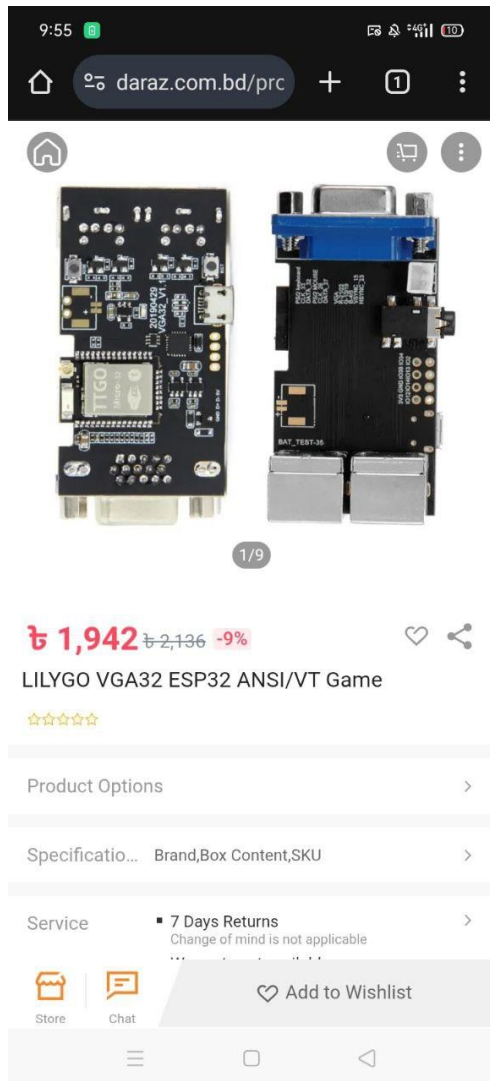
Component	Current Cost	Discount Rate (%)	Discounted cost
ESP32S3 N16R8 (WROOM1)	1300	10	1170
PCB Board	1300	20	1040
5V AC/DC power supplier adapter	80	10	72
Capacitor 470 μ F	165	10	150
Resistors (470 Ω)	38	0	38
Resistors (220 Ω)	32	0	32
VGA DB15 Connector	30	0	30
DC Power Jack Socket Connector	10	0	10
VGA Cable	200	10	180
Miscellaneous cost	15	---	63
Total			2800

8.4 Timeline of Project Implementation ^{COS}



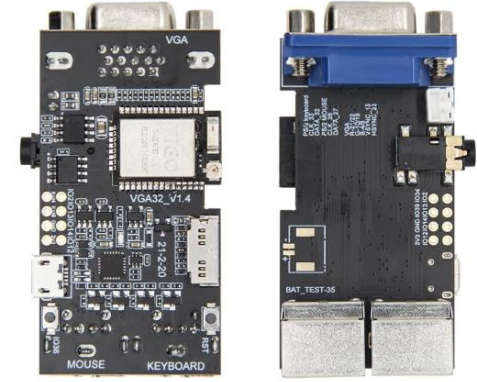
9 Comparing with similar products ^{CO7}

9.1 Our First Competitor:



- Limited to 320x240 resolution
- ESP32 has limited RAM, especially for high-res images
- Must optimize image size and format
- Wi-Fi + VGA rendering both consume memory, so careful buffer management is needed
- In April/May, 2025 we found it out of the stock. But now it is nowhere in the market.
- Using such compact architecture with SMD devices may reduce our prototype cost significantly also.

9.2 Our Closest Existing Competitor:



LILYGO

LILYGO FabGL VGA32 VGA ESP32 PSRAM Module Controller PS/2 Mouse Keyboard Graphics Library Game ANSI/VT Terminal Circuits TTGO Development Board

99% of respondents would recommend this to a friend

Item #: 83928684

BDT 4509 ~~BDT-4819~~ You Save : 6% [PRICE DROP](#)

★★★★★ 5.0 rating [Write a review](#)

Availability : In stock ✓  Imported from USA store

U-Care Warranty:

Powerful and versatile functionality may make the cost of this module high.

10 Future Work

While the system is already functional and meets its core objectives, there are several enhancements planned to improve its real-time performance and deployment reliability. Currently, we are hosting the backend Python script on railway.app, which is sufficient for demonstration but has limitations in uptime and flexibility. Due to the absence of a credit card, we were unable to use **Google Cloud VM** or **Firebase**, which would offer more robust and reliable hosting.

In the future, we plan to:

- **Migrate the backend to Google Cloud VM or Firebase** for improved execution speed, uptime, and reduced CDN-related delays.
- **Deploy the system permanently in the ECE Seminar Room** during the upcoming term break, making it a real-time, production-ready solution.

These improvements aim to make the system more scalable, professional, and suitable for broader institutional use.

11 Appendix:

Master Code:

```
#include <Arduino.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <SPI.h>
#include <time.h>
#include <esp_wifi.h>

// ===== SPI Pin Assignments =====
#define PIN_MOSI 9
#define PIN_SCLK 10
#define PIN_CS 11
#define PIN_READY 12
#define PIN_ACK 13

// ===== Constants =====
#define CHUNK_SIZE 1024
#define TOTAL_CHUNKS 600
#define TOTAL_SIZE (CHUNK_SIZE * TOTAL_CHUNKS)
#define WIFI_TIMEOUT 30000 // 30 seconds
#define CYCLE_DELAY 30000 // 30 seconds
#define DOWNLOAD_RETRY_DELAY 5000
#define MAX_DOWNLOAD_ATTEMPTS 3

const char* ssid = "Horseman"; //change
const char* password = "bme.eee.buet.ac.bd"; //change

// ===== GitHub Raw URLs =====
const char* versionURL = "https://raw.githubusercontent.com/Sazid-121/esp32-vga-notification-board/main/version.txt";
const char* binURL = "https://raw.githubusercontent.com/Sazid-121/esp32-vga-notification-board/main/display_image_rgb565.bin";

// ===== Globals =====
uint8_t* binData = nullptr;
int previous_version = -1;

// ===== Functions =====

void initSPIpins() {
  pinMode(PIN_READY, OUTPUT);
  pinMode(PIN_ACK, INPUT_PULLDOWN);
  pinMode(PIN_CS, OUTPUT);
  digitalWrite(PIN_READY, LOW);
  digitalWrite(PIN_CS, HIGH);
  SPI.begin(PIN_SCLK, -1, PIN_MOSI, PIN_CS);
  Serial.println("✅ SPI initialized");
}

void connectToWiFi() {
  WiFi.mode(WIFI_STA);
  esp_wifi_set_max_tx_power(80);

  WiFi.begin(ssid, password);
  Serial.print("📶 Connecting to Wi-Fi");
```



```

while (WiFi.status() != WL_CONNECTED) {
  Serial.print(".");
  delay(500);
}

Serial.println("\n✅ Connected to Wi-Fi.");
}

bool isWiFiConnected() {
  return WiFi.status() == WL_CONNECTED;
}

bool fetchVersion(int &version, String &renderTime) {
  Serial.println("⬇ Downloading version.txt...");

  for (int attempt = 1; attempt <= MAX_DOWNLOAD_ATTEMPTS; attempt++) {
    if (!isWiFiConnected()) {
      Serial.println("🔄 Wi-Fi disconnected. Reconnecting...");
      connectToWiFi();
    }

    HTTPClient http;
    http.begin(versionURL);
    int httpCode = http.GET();

    if (httpCode == HTTP_CODE_OK) {
      String payload = http.getString();
      http.end();

      Serial.println("📄 version.txt content:");
      Serial.println(payload);

      payload.trim();
      int lineBreak = payload.indexOf("\n");
      if (lineBreak == -1) {
        Serial.println("❌ Malformed version.txt");
        return false;
      }

      version = payload.substring(0, lineBreak).toInt();
      renderTime = payload.substring(lineBreak + 1);
      renderTime.trim();

      Serial.printf("📄 version: %d | render_time: %s\n", version, renderTime.c_str());
      return true;
    }

    http.end();
    Serial.printf("❌ Attempt %d: Failed to download version.txt (code %d)\n", attempt, httpCode);
    if (attempt < MAX_DOWNLOAD_ATTEMPTS) delay(DOWNLOAD_RETRY_DELAY);
  }

  Serial.println("❌ Failed to fetch version.txt after retries. Skipping cycle.");
  return false;
}

bool downloadBinFile() {
  Serial.println("⬇ Downloading .bin file...");

  for (int attempt = 1; attempt <= MAX_DOWNLOAD_ATTEMPTS; attempt++) {

```

```

if (!isWiFiConnected()) {
    Serial.println("🔌 Wi-Fi disconnected. Reconnecting...");
    connectToWiFi();
}

HTTPClient http;
http.begin(binURL);
int httpCode = http.GET();

if (httpCode == HTTP_CODE_OK) {
    int size = http.getSize();
    Serial.printf("📁 .bin file size: %d bytes\n", size);

    if (size != TOTAL_SIZE) {
        Serial.printf("⚠️ Expected %d bytes, got %d\n", TOTAL_SIZE, size);
    }

    binData = (uint8_t*)ps_malloc(TOTAL_SIZE);
    if (!binData) {
        Serial.println("❌ PSRAM allocation failed.");
        http.end();
        return false;
    }

    WiFiClient *stream = http.getStreamPtr();
    int index = 0;
    while (http.connected() && index < size) {
        if (stream->available()) {
            binData[index++] = stream->read();
        }
    }

    http.end();
    Serial.println("✅ .bin file download complete.");
    return true;
}

http.end();
Serial.printf("❌ Attempt %d: Failed to download .bin file (code %d)\n", attempt, httpCode);
if (attempt < MAX_DOWNLOAD_ATTEMPTS) delay(DOWNLOAD_RETRY_DELAY);
}

Serial.println("❌ Failed to fetch .bin file after retries. Skipping cycle.");
return false;
}

void waitUntilRenderTime(const String& renderTimeStr) {
    if (renderTimeStr == "now") {
        Serial.println("🕒 Render time is NOW. Proceeding immediately.");
        return;
    }
}

struct tm timeinfo;
if (!strptime(renderTimeStr.c_str(), "%Y-%m-%dT%H:%M:%S", &timeinfo)) {
    Serial.println("❌ Failed to parse render time. Rendering now.");
    return;
}

time_t targetTime = mktime(&timeinfo);
time_t now;

```

```

time(&now);

while (now < targetTime) {
    int remaining = targetTime - now;
    Serial.printf("⌚ Waiting %d seconds until render time...\n", remaining);
    delay(min(10000, remaining * 1000));
    time(&now);
}

Serial.println("✅ Render time reached. Proceeding...");
}

void sendRawFramebuffer() {
    Serial.println("📺 Sending framebuffer to slave...");

    for (uint16_t chunk = 0; chunk < TOTAL_CHUNKS; chunk++) {
        digitalWrite(PIN_READY, HIGH);
        while (digitalRead(PIN_ACK) == LOW) delay(1);

        while (digitalRead(PIN_CS) == LOW) delay(1); // wait for CS to be HIGH
        digitalWrite(PIN_CS, LOW);

        SPI.beginTransaction(SPISettings(100000, MSBFIRST, SPI_MODE0));
        for (int i = 0; i < CHUNK_SIZE; i++) {
            SPI.transfer(binData[chunk * CHUNK_SIZE + i]);
        }
        SPI.endTransaction();
        digitalWrite(PIN_CS, HIGH);

        while (digitalRead(PIN_ACK) == HIGH) delay(1);
        digitalWrite(PIN_READY, LOW);
        delay(2);
    }

    Serial.println("✅ Framebuffer sent.");
    free(binData);
    binData = nullptr;
}

// ===== SETUP =====
void setup() {
    Serial.begin(115200);
    delay(500);
    Serial.println("💖 Booting ESP32-S3 MASTER...");

    initSPIpins();
    connectToWiFi();
    configTime(21600, 0, "pool.ntp.org"); // UTC+6
}

// ===== LOOP =====
void loop() {
    if (!isWiFiConnected()) {
        Serial.println("🔌 Wi-Fi disconnected. Reconnecting...");
        connectToWiFi();
    }

    Serial.println("🔄 Checking version.txt...");

    int currentVersion;

```

```

String renderTime;

if (fetchVersion(currentVersion, renderTime)) {
  if (currentVersion != previous_version) {
    Serial.println("📦 New version detected. Downloading .bin...");

    if (downloadBinFile()) {
      waitUntilRenderTime(renderTime);
      sendRawFramebuffer();
      previous_version = currentVersion;
    }
  } else {
    Serial.println("⏸ No update. Version unchanged.");
  }
}

Serial.println("⌚ Sleeping 30 seconds...\n");
delay(CYCLE_DELAY);
}

```

Slave Code:

```
#include <Arduino.h>
#include <driver/spi_slave.h>
#include <esp_heap_caps.h>
#include "luniVGA.h" // Bitluni VGA library

// ===== Pin Definitions =====
#define PIN_MOSI 42
#define PIN_SCLK 41
#define PIN_CS 40
#define PIN_READY 39 // Master → Slave
#define PIN_ACK 38 // Slave → Master

#define SPI_SLAVE_HOST SPI3_HOST
#define CHUNK_SIZE 1024
#define TOTAL_CHUNKS 600
#define FRAMEBUFFER_SIZE (CHUNK_SIZE * TOTAL_CHUNKS)
#define TIMEOUT_MS 3000

// VGA Pin config (Do NOT overlap with SPI)
const PinConfig pins(
    14, 13, 12, 11, 10, // Red
    9, 8, 18, 17, 16, 15, // Green
    21, 7, 6, 5, 4, // Blue
    2, 1 // HSYNC, VSYNC
);

uint8_t* framebuffer_psram = nullptr;
uint8_t* rx_buffer = nullptr;

void drawImageFromBuffer(uint8_t* buffer) {
    const int IMAGE_WIDTH = 640;
    const int IMAGE_HEIGHT = 480;

    for (int y = 0; y < IMAGE_HEIGHT; y++) {
        for (int x = 0; x < IMAGE_WIDTH; x++) {
            int i = y * IMAGE_WIDTH + x;
            uint16_t rgb565 = ((uint16_t*)buffer)[i];

            uint8_t r = ((rgb565 >> 11) & 0x1F) << 3;
            uint8_t g = ((rgb565 >> 5) & 0x3F) << 2;
            uint8_t b = (rgb565 & 0x1F) << 3;

            vgaDot(x, y, r, g, b);
        }
    }

    vgaShow();
    vgaStart();
}

void blankTask() {
    const int WIDTH = 640, HEIGHT = 480;

    for (int y = 0; y < HEIGHT; y++) {
        for (int x = 0; x < WIDTH; x++) {
            vgaDot(x, y, 0, 0, 0);
        }
    }
}
```

```

    }
}

vgaShow();
vgaStart();
}

void setup() {
    Serial.begin(115200);
    delay(1000);
    Serial.println("🟢 Slave: Receiving 600 chunks without chunkID");

    pinMode(PIN_READY, INPUT_PULLDOWN);
    pinMode(PIN_ACK, OUTPUT);
    digitalWrite(PIN_ACK, LOW);

    // VGA init
    VGAMode mode = MODE_640x480x60;
    if (!vgaInit(pins, mode, 16)) {
        Serial.println("❌ VGA init failed");
        while (true);
    }
    vgaStart();
    framebuffer_psram = (uint8_t*)heap_caps_malloc(FRAMEBUFFER_SIZE,
MALLOC_CAP_SPIRAM);
    rx_buffer = (uint8_t*)heap_caps_malloc(CHUNK_SIZE, MALLOC_CAP_DMA);

    if (!framebuffer_psram || !rx_buffer) {
        Serial.println("❌ Memory allocation failed");
        while (true);
    }

    memset(framebuffer_psram, 0, FRAMEBUFFER_SIZE);

    spi_bus_config_t buscfg = {
        .mosi_io_num = PIN_MOSI,
        .miso_io_num = -1,
        .sclk_io_num = PIN_SCLK,
        .quadwp_io_num = -1,
        .quadhd_io_num = -1,
        .max_transfer_sz = CHUNK_SIZE
    };

    spi_slave_interface_config_t slvcfg = {
        .spics_io_num = PIN_CS,
        .flags = 0,
        .queue_size = 1,
        .mode = 0,
        .post_setup_cb = NULL,
        .post_trans_cb = NULL
    };

    if (spi_slave_initialize(SPI_SLAVE_HOST, &buscfg, &slvcfg, SPI_DMA_CH_AUTO) != ESP_OK)
    {
        Serial.println("❌ SPI init failed");
        while (true);
    }

    Serial.println("✅ SPI initialized. Waiting for chunks...");
}

```

```

}

void loop() {
    static uint16_t receivedChunks = 0;
    static bool done = false;

    //if (done) return;

    if (digitalRead(PIN_READY) == LOW)
    {
        return;
    }

    if (done) {
        if (digitalRead(PIN_READY) == HIGH) {
            ESP.restart();
        }
    }

    digitalWrite(PIN_ACK, HIGH);

    spi_slave_transaction_t t;
    memset(&t, 0, sizeof(t));
    t.length = CHUNK_SIZE * 8;
    t.rx_buffer = rx_buffer;

    esp_err_t ret = spi_slave_transmit(SPI_SLAVE_HOST, &t, pdMS_TO_TICKS(TIMEOUT_MS));

    digitalWrite(PIN_ACK, LOW);
    while (digitalRead(PIN_READY) == HIGH) delay(1);

    if (ret == ESP_OK) {
        // Store chunk by arrival order
        memcpy(&framebuffer_psr[receivedChunks * CHUNK_SIZE], rx_buffer, CHUNK_SIZE);
        Serial.printf("✅ Chunk %u received\n", receivedChunks);
        Serial.print("🔍 Received (first 8 bytes): ");
        for (int i = 0; i < 8; i++) {
            Serial.printf("%02X ", rx_buffer[i]);
        }
        Serial.println();

        Serial.println();
        receivedChunks++;

        if (receivedChunks == TOTAL_CHUNKS) {
            Serial.println("🎉 All chunks received.");

            drawImageFromBuffer(framebuffer_psr);

            Serial.println("##### VGA DRAWN #####");
            //delay(10000);
            done = true;
        }
    } else {
        Serial.printf("⚠️ SPI error: %s\n", esp_err_to_name(ret));
    }
}

```